

Chapter Overview

Chapter 19 covers DevOps and CI/CD for ShopVerse: GitHub Actions pipeline, Docker image build, Kubernetes deployment via GitOps, secret management (Vault), observability stack (Prometheus, Tempo, Loki, Grafana), SLIs/SLOs/Error Budgets, feature flags, blue-green & canary deployments, chaos engineering, and a full end-to-end release workflow from commit to production.

Sub-Chapter	Topic
19-01	DevOps Culture & DORA Metrics – shift-left, deployment frequency
19-02	GitHub Actions CI Pipeline – tests, SAST, Trivy, SonarCloud
19-03	Docker Build in CI – SHA tags, multi-arch, BuildKit GHA cache
19-04	Kubernetes CD – Helm + ArgoCD GitOps sync
19-05	Secret Management – Vault, External Secrets Operator
19-06	Observability – Prometheus, Grafana Tempo, Loki
19-07	SLIs, SLOs & Error Budgets – reliability contracts
19-08	Feature Flags – LaunchDarkly, progressive rollout
19-09	Blue-Green Deployment – zero-downtime switch
19-10	Canary Deployment – traffic splitting, auto-rollback
19-11	Chaos Engineering – fault injection, pod kill experiments
19-12	Full Release Workflow – commit to production
19-13	Post-Deploy Monitoring & Runbooks

DORA Metric	Elite	High	Medium	Low (lagging)
Deployment Frequency	Multiple/day	Weekly	Monthly	Every 6 months
Lead Time for Change	< 1 hour	1 day – 1 week	1 week – 1 month	> 6 months
Change Failure Rate	< 5%	< 15%	< 30%	> 30%
Time to Restore Service	< 1 hour	< 1 day	< 1 week	> 1 week

Shift-Left Pipeline – defects found earlier = cheaper to fix

Commit → PR → Merge → Deploy – cost of defect rises right to left

Unit Tests on every commit (< 30s) – catches logic bugs

Static Analysis: Checkstyle, SpotBugs, SonarQube – code quality

SAST (Semgrep, CodeQL) – security vulnerabilities in source code

Container Image Scan (Trivy) – CVEs in base image + dependencies

Contract Tests (Pact) – API compatibility before merge

Integration Tests (TestContainers) – real infra, no mocks

Load Tests (Gatling) – P95 < 500ms before release

Smoke Tests in production – verify live endpoint post-deploy

```
# .github/workflows/ci.yml
name: ShopVerse CI
on: { push: { branches: [main] }, pull_request: {} }
jobs:
test:
runs-on: ubuntu-latest
services:
postgres: { image: postgres:16, env: { POSTGRES_PASSWORD: test } }
redis: { image: redis:7-alpine }
steps:
- uses: actions/checkout@v4
- uses: actions/setup-java@v4
with: { java-version: "21", distribution: "temurin" }
- uses: actions/cache@v4
with: { path: ~/.m2, key: maven-${{ hashFiles("**/pom.xml") }} }
- run: ./mvnw verify # unit + integration + JaCoCo coverage
- uses: aquasecurity/trivy-action@master
with: { scan-type: fs, severity: CRITICAL,HIGH, exit-code: 1 }
- uses: SonarSource/sonarcloud-github-action@master
env: { SONAR_TOKEN: ${ secrets.SONAR_TOKEN } }
```

CI Stage	Tool	Failure Action
Unit + Integration tests	JUnit 5 + TestContainers	Block PR merge; link to failed test report
Code coverage gate	JaCoCo (80% minimum)	Block merge if coverage drops
Static analysis	SonarCloud, SpotBugs	Block on Critical/Blocker issues
SAST scan	Trivy fs / Semgrep	Block on CRITICAL/HIGH CVE in source

```
# .github/workflows/docker.yml
jobs:
  build-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-qemu-action@v3 # multi-arch: amd64 + arm64
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
    with: { registry: ghcr.io, username: ${ github.actor },
    password: ${ secrets.GITHUB_TOKEN } }
      - uses: docker/build-push-action@v5
    with:
      push: true
      platforms: linux/amd64,linux/arm64
      tags: |
        ghcr.io/shopverse/product-service:${ github.sha }
        ghcr.io/shopverse/product-service:${ github.ref_name }
      cache-from: type=gha
      cache-to: type=gha,mode=max
      provenance: true # SLSA build provenance attestation
      sbom: true # Software Bill of Materials
      - uses: aquasecurity/trivy-action@master
    with: { image-ref: ghcr.io/shopverse/product-service:${ github.sha },
    severity: CRITICAL,HIGH, exit-code: 1 }
```

```
# CD: update Helm values with new SHA → push to Git → ArgoCD syncs
- name: Update image tag
run: |
yq e ".image.tag = \"${{ github.sha }}\"" -i helm/values-prod.yaml
git commit -am "ci: product-service → ${{ github.sha }}"
git push
- name: Wait for ArgoCD
run: argocd app wait product-service --timeout 300 --health
- name: Smoke test
run: curl -f https://api.shopverse.com/api/v1/products/health || exit 1
```

ArgoCD Application

```
spec:
source:
repoURL: https://github.com/shopverse/k8s-config
path: apps/product-service
syncPolicy:
automated: { prune: true, selfHeal: true }
```

CD Stage	Tool	Time Budget	Gate
Update Helm values in Git	git push	< 30s	Commit succeeds
ArgoCD rolling update	ArgoCD + K8s Deployment	< 5 min	All pods pass readiness probe
Wait for sync	argocd app wait	< 5 min	App health = Healthy
Smoke test	curl /health endpoint	< 30s	HTTP 200 from production

HashiCorp Vault – central secret store + rotation

Vault KV v2: secret/shopverse/product-service → { db-password, jwt-secret }

Option A – Vault Agent Injector (sidecar)

Annotation injects secret file into /vault/secrets/ at pod start

Spring reads via @Value("\${db.password}") from file

Option B – External Secrets Operator (ESO)

ExternalSecret CR: sync Vault secret → K8s Secret

Refreshes automatically every 1h; no pod restart needed

Pod reads K8s Secret as env var or volume mount

```
# ExternalSecret: sync Vault → K8s Secret every 1 hour
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata: { name: db-creds, namespace: shopverse }
spec:
  refreshInterval: 1h
  secretStoreRef: { name: vault-backend, kind: ClusterSecretStore }
  target: { name: db-secret }
  data:
    - secretKey: password
  remoteRef: { key: shopverse/product-service, property: db-password }
```

Observability Stack – ShopVerse (Grafana LGTM)

Metrics – Prometheus + Grafana

Actuator /actuator/prometheus → Prometheus scrapes every 15s

JVM metrics (GC, heap), HTTP request rate, HikariCP pool, Kafka lag

Distributed Tracing – OpenTelemetry + Grafana Tempo

Micrometer Tracing → OTLP → Tempo; TraceId in MDC log field

sampling.probability=0.1 in prod (10% sampled)

Logging – Logback JSON → Promtail → Grafana Loki

JSON log: { traceId, spanId, level, service, message, timestamp }

Loki labels: namespace, pod; query in Grafana via LogQL

Grafana – unified: link trace → logs → metrics in one click

Signal	Tool	Key Metric / Config
Metrics	Prometheus + Grafana	http_request_duration_seconds P95; Kafka consumer lag
Tracing	OpenTelemetry + Tempo	sampling 10% prod; TraceId links to Loki log
Logging	Loki + Promtail	JSON; include traceId in MDC; LogQL queries in Grafana

Term	Definition	ShopVerse Example
SLI	Metric measuring service health	% product searches returning 200 in < 500ms
SLO	Target value for SLI	99.5% searches complete in < 500ms over 30 days
Error Budget	100% minus SLO	0.5% of searches may fail or be slow per month
SLA	Contractual SLO commitment	99% uptime; financial penalty if breached
Service	SLO	Error Budget / 30 days
product-service	99.5% < 500ms; 99.5% success	0.5% = ~216 minutes of degraded service
order-service	99.9% order success	0.1% = ~43 minutes of failures
payment-service	99.99% payment success	0.01% = ~4 minutes – very tight
api-gateway	99.95% non-5xx responses	0.05% = ~22 minutes of gateway errors

```
# Prometheus alert: error budget burn rate > 5x normal
- alert: HighErrorBudgetBurnRate
expr: |
(1 - sum(rate(http_requests_total{status!="2.."}[1h]))
/ sum(rate(http_requests_total[1h]))) > 0.025
annotations: { summary: "Error budget burning 5x fast - check dashboards" }
```



```

@RestController
public class ProductController {
    @Autowired private LDClient launchDarkly;
    @GetMapping("/api/v1/products/search")
    public SearchResult search(@RequestParam String q, @AuthUser User user) {
        boolean useNewRanking = launchDarkly.boolVariation(
            "new-ranking-algorithm",
            new LDUser(String.valueOf(user.getId())),
            false); // default: feature off
        return useNewRanking
            ? newRankingService.search(q)
            : legacyService.search(q);
    }
}

```

Progressive Rollout Strategy

Phase	Target	Traffic	Monitor
1 – Internal	shopverse.com email users	0 → 5%	Latency, errors, no customer impact
2 – Beta	Opted-in beta segment	5 → 25%	A/B: new vs old conversion rate
3 – General	All users	25 → 100%	Full SLO monitoring
Rollback	Any phase	Flip flag to 0% instantly	Instant; no deployment needed

Blue-Green – Instant Switch

Blue-Green Deployment

- Blue (v1.4) – current production; 100% traffic via LB/DNS
- Green (v1.5) – new version deployed; integration + smoke tests run
- Switch: load balancer weight → 100% Green (DNS flip or K8s Service selector)
- Zero downtime; instant; Green = production
- Keep Blue for 30 min → rollback = flip back in seconds if issues

Canary – Gradual Traffic Split (Istio)

```
# 10% traffic to v2; 90% to v1
spec:
  http:
    - route:
      - destination: { host: product-service, subset: v1 } weight: 90
      - destination: { host: product-service, subset: v2 } weight: 10
# Automatic rollback if P95 > 500ms or error rate > 1%:
# Shift weight back to 100% v1 immediately
```

Strategy	Traffic Shift	Rollback Speed	Blast Radius
Blue-Green	Instant (all at once)	Instant DNS/LB switch	All users on new version
Canary	Gradual % (5→10→50→100)	Shift weight to 0% in 10 minutes	Only % of users on new version
Feature Flag	Per-user in code path	30 seconds (flip flag)	Configurable % of users

Experiment	Hypothesis	Method	Expected Outcome
Kill 1 of 3 pods	HPA replaces; 2 pods serve all traffic	kubectl delete pod random	No 5xx; HPA scales back to 3 within 60s
500ms latency to inventory-service	Circuit breaker opens; fallback to cache	istioctl inject --delay 500ms --percentage 30	CB opens after 3 failures; cache -1 returned
Block Postgres primary	Replica promoted; reconnects	psql -h postgres -c 'DROP to postgres primary'	Primary promotes replica; JDBC reconnects
Exhaust HikariCP pool	3000ms timeout triggers; 503s fail fast	503s fail fast	connection-timeout=3s fail fast; not indefinite hang

```
# Istio - inject 500ms delay to 30% of inventory requests
apiVersion: networking.istio.io/v1
kind: VirtualService
spec:
  http:
  - fault:
      delay: { fixedDelay: 500ms, percentage: { value: 30 } }
    route: [{ destination: { host: inventory-service } }]
```

Commit → Production in < 20 Minutes

Developer merges PR to main

GitHub Actions CI triggers:

1. `./mvnw verify` → unit + integration (TestContainers) < 8 min
2. Trivy fs scan → CRITICAL/HIGH CVE fails build
3. SonarCloud analysis → gate must pass

GitHub Actions CD triggers (after CI green):

4. docker build + push SHA tag to GHCR < 5 min
5. Trivy image scan → fail on CRITICAL/HIGH
6. yq updates helm/values-prod.yaml; git push to k8s-config repo

ArgoCD auto-sync (detects Git diff):

7. helm upgrade → K8s rolling update (maxUnavailable:0, maxSurge:1)
8. readiness probe gates; old pod terminates after new is ready

Post-deploy:

9. Smoke test: `curl /api/v1/products/health` → 200 OK
10. Monitor: Grafana error rate dashboard for 10 min

Monitoring Check	Tool	Action if Alert Fires
P95 latency spikes above 500ms	Prometheus/Grafana	Check trace waterfall in Tempo; identify slow service
Error rate > 1% (5xx)	Prometheus alert	Immediate rollback: argocd app rollback; then investigate
Pod CrashLoopBackOff	K8s events + Loki logs	kubectl logs -p (previous); check OOM or startup error
Kafka consumer lag grows	Kafka Exporter + Grafana	Scale consumer deployment; check slow consumer log
HikariCP pool near exhaustion	Monitor metric	Check slow queries in pg_stat_statements; add index

Runbook Template

```
# Runbook: product-service P95 > 500ms
## Detection: Grafana alert "ProductSearchP95High"
## Impact: User searches slow; potential conversion drop
## Steps:
1. Check Grafana trace: Tempo → find slow trace → identify bottleneck service
2. If Elasticsearch slow: check ES cluster health GET /_cluster/health
3. If DB slow: check pg_stat_statements; look for sequential scans
4. If new deployment: argocd app rollback product-service
5. If HikariCP: reduce active connections; add read replica
## Escalation: if not resolved in 30 min → page on-call architect
```

Alert Severity	Response Time	Escalation
P1 (complete outage)	5 minutes	On-call SRE + architect + CTO if > 15 min
P2 (degraded)	15 minutes	On-call SRE; escalate if > 30 min
P3 (performance)	1 hour	Next business hour; track in Jira

■ Do

- ✓ Measure DORA metrics on the team dashboard – deployment frequency and MTTR are leading indicators.
- ✓ Run all gates in CI: tests, coverage, SAST, image scan before any merge to main.
- ✓ Use GitOps (ArgoCD): Git commit = deploy trigger; Git history = full audit trail; easy rollback.
- ✓ Tag images with git SHA; use SHA in K8s Deployments – immutable; no surprise on pull.
- ✓ Use Vault/External Secrets Operator for production secrets – rotation without pod restart.
- ✓ Define SLOs before going live; alert on error budget burn rate > 5× normal.
- ✓ Use feature flags for risky features – decouple code deployment from feature release.
- ✓ Run chaos experiments in staging regularly – validate circuit breakers, HPA, Patroni failover.
- ✓ Run post-deploy smoke tests – verify production health before closing the CD pipeline.
- ✓ Write runbooks for every alert – MTTR drops dramatically when steps are pre-documented.

■ Anti-Patterns

- Manual kubectl apply in production – no audit trail; state drifts from Git.
- Secrets in Git (even encrypted SOPS without rotation) – use Vault for dynamic secrets.
- Deploying all microservices simultaneously – single large blast radius; impossible to isolate.
- No rollback plan – after a bad deploy, how long to restore? Target < 5 minutes.
- Canary testing only (no CI test suite) – canary finds production bugs, not pre-production.
- No observability before go-live – no metrics = flying blind; cannot enforce SLOs.
- Ignoring DORA metrics – team may feel busy but lead time and CFR reveal actual delivery health.

Sub-Chapter	Key Point	One-Liner
19-01 DORA	Elite: multi deploys/day; CRF < 1% < 1h; MTTR < 1h at elite level	Lead < 5m
19-02 CI	Test + SAST + Trivy on every commit	Block merge on CRITICAL CVE or test failure
19-03 Docker	SHA tag + GHA cache	Multi-arch; SBOM; post-push image scan
19-04 CD ArgoCD	CD → git push → auto-sync	argocd app wait 300s; smoke test
19-05 Vault	ExternalSecret refreshInterval	K8s Secret auto-updated; no restart needed
19-06 Observability	Prometheus + Tempo + Loki	Grafana links trace → log → metric
19-07 SLO	Error budget = 100% - SLO	Burn rate alert; freeze deploys if budget exhausted
19-08 Feature Flags	Progressive % rollout	Flip to 0% = 30s rollback; no deployment
19-09 Blue-Green	Keep Blue 30 min for rollback	Instant switch; full blast radius
19-10 Canary	10% traffic; auto-rollback	Shift weight to 0% on P95 regression
19-11 Chaos	Fault injection in staging	Validate CB, Patroni, HPA, HikariCP timeout
19-12 Full Flow	Commit → prod < 20 min	Test → Build → Scan → GitOps → Smoke
19-13 Runbooks	Re-documented steps	MTTR drops significantly with written runbooks

Interview Quick-Check

Question	Concise Answer
DORA metrics?	Deployment Frequency, Lead Time, Change Failure Rate, Time to Restore. Elite: multiple deploys/day
GitOps vs push-based CD?	GitOps: Git = source of truth; ArgoCD detects drift and self-heals; audit trail = Git history. Push: CI runs
SLO vs SLA?	SLO: internal reliability target (e.g. 99.5% search < 500ms). SLA: contractual commitment with financial
Canary vs Blue-Green?	Blue-Green: maintain 2 envs; instant switch; full blast radius. Canary: gradual % shift; limited blast radius
Feature flag benefit?	Decouples deployment from release. Instant rollback (flip flag, no deploy). Progressive rollout (5% →